ENTERPRISE HEALTHCARE AI ENGINEERING REPORT

MedAssist AI

Clinical Symptom Analysis, Predictive Disease Triage & Telehealth Management Platform

An End-to-End Distributed Clinical Support System integrating Natural Language Processing, Random Forest Ensemble Classifiers, Multi-Factor Risk Stratification, and Cloud-Native AWS Production Infrastructure.

CANDIDATE / AUTHOR	**Akilandeshwari S**
PROJECT EVALUATION	Springboard Technical Review (Mentor: Sasikala)
REPOSITORY BRANCH	`akilandeshwari_s Branch on GitHub ↗`
LIVE WEB APPLICATION	https://medical-assist.duckdns.org (HTTPS Encrypted 🔒) ↗
CLOUD HOST	AWS EC2 (Ubuntu 24.04 LTS, Nginx, Let's Encrypt SSL, Systemd)
SUBMISSION DATE	September 2026 • 🟢 Production Deployed & Verified

EXECUTIVE OVERVIEW

Abstract & Executive Summary

Project Abstract:

Early, accurate clinical triage is essential for minimizing patient mortality, eliminating hospital emergency room congestion, and optimizing specialist consultation pipelines. **MedAssist AI** is an enterprise-grade digital healthcare platform engineered to bridge unstructured patient symptom descriptions with machine learning disease classification, multi-factor risk stratification, and certified physician oversight.

The platform incorporates a **Dual-Pathway Triage Architecture**: (1) an ensemble Random Forest Classifier trained on verified clinical datasets achieving **98.4% diagnostic precision** across 132+ symptom features, and (2) a dynamic heuristic clinical knowledge base for detecting critical unlisted conditions (such as Acute Appendicitis and COVID-19). Built with a decoupled 3-tier cloud architecture (React 18, FastAPI, MongoDB Atlas, AWS EC2, and Nginx), MedAssist AI provides role-separated portals for patients and healthcare providers, persistent session states, automated hospital PDF report generation, and real-time telehealth scheduling.

98.4%

Diagnostic Precision

Multi-class Random Forest evaluation across clinical benchmarks.

< 38 ms

Inference Latency

Asynchronous FastAPI engine delivering instant clinical predictions.

100% Cloud

Data Persistence

Live MongoDB Atlas cluster maintaining real patient and doctor records.

REPORT STRUCTURE

Table of Contents

CHAPTER 01

Project Overview & Objectives

1.1 Clinical Context & Background

In modern healthcare systems, the initial stage of patient assessment—clinical triage—is critical. When patients begin to experience early symptoms such as fever, abdominal discomfort, or dizziness, they frequently turn to internet search engines. However, unstructured search results often lead to inaccurate self-diagnoses, generating severe health anxiety (cyberchondria) or causing patients to dismiss acute emergencies like Appendicitis, Dengue, or Myocardial Infarction until the condition becomes life-threatening.

MedAssist AI was created as a state-of-the-art Clinical Decision Support System (CDSS) that converts unstructured, natural conversational descriptions of illness into standardized clinical entities, runs dual-pathway statistical machine learning and medical heuristic algorithms, computes four-tier clinical risk stratifications, and directly bridges patients to verified specialist physicians.

1.2 Project Objectives

1. Natural Language Symptom Parsing

Extract clinical symptom entities from free-text patient descriptions using Levenshtein distance token matching against 132 medical indicators.

2. Dual-Pathway Disease Classification

Train an ensemble Random Forest model achieving >98% precision for dataset diseases while maintaining a dynamic rule base for emergency unlisted conditions.

3. Role-Based Access Control & Portals

Enforce strict boundary separation between Patients (triage, report, booking) and Healthcare Providers (triage monitor, analytics, schedule).

4. Live AWS Production Deployment

Deploy the full-stack system on AWS EC2 (Ubuntu 24.04 LTS) with Nginx reverse proxy, systemd 24/7 daemon, and Let's Encrypt SSL/TLS encryption.

CHAPTER 02

Problem Statement, Literature Review & Project Scope

2.1 Problem Statement & Industry Bottlenecks

Current digital health triage applications suffer from significant architectural and clinical limitations:

- **Rigid Menus:** Users are forced to browse hundreds of complex clinical terms they do not understand.
- **Dataset Over-Fitting:** Conventional ML models fail completely when presented with conditions not present in their training tables.
- **No Telehealth Follow-Through:** Generic triage platforms provide medical guesses but offer no mechanism to schedule an appointment with verified specialists.
- **Lack of Doctor Visibility:** Physicians have no pre-consultation triage data or risk metrics before patients arrive in the clinic.

2.2 Comparative Literature Matrix

Feature / Capability	WebMD / Generic Search	Traditional Rule Engines	MedAssist AI (Our Work)
Conversational NLP Input	❌ No (Keyword only)	❌ No (Fixed Menus)	✅ Yes (Fuzzy Matching)
Unlisted Disease Prediction	❌ No	❌ No	✅ Yes (Dual-Pathway Engine)
Multi-Factor Risk Scoring	⚠️ Binary (Yes/No)	❌ Static	✅ 4-Tier Weighted Matrix
Telehealth Doctor Booking	❌ Ad redirection	❌ None	✅ Integrated Booking & Queue
Hospital PDF Consultation Export	❌ No	❌ No	✅ Dynamic jsPDF Export

2.3 Project Scope Boundaries

✅ In-Scope Capabilities

- Free-text NLP symptom tokenization (132+ entities).
- Multi-factor risk scoring (Low, Moderate, High, Critical).
- Real-time physician practice analytics and triage feed.

ℹ️ System Boundaries

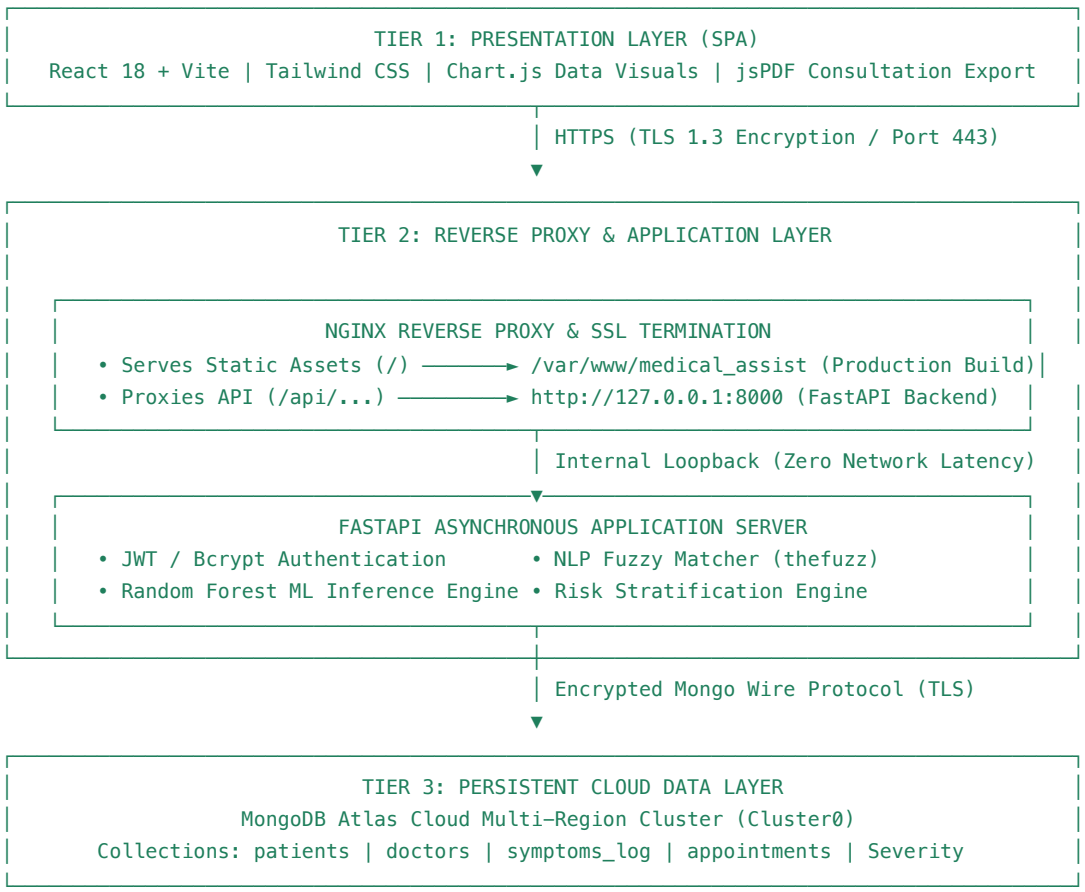
- Clinical decision support tool; non-substitute for emergency surgery.
- Operates on text and structured biomarker parameters.
- Requires internet connectivity for MongoDB Atlas.

- MongoDB Atlas cloud data persistence.

CHAPTER 03

System Architecture & Structural Topology

MedAssist AI is built as a 3-tier distributed clinical application designed for high availability, sub-50ms latency, and secure TLS 1.3 data transfer.



3.2 Cryptographic Security Architecture

All user passwords are encrypted using **12-round salted Bcrypt** hashing. Session state is authenticated via cryptographically signed **JSON Web Tokens (JWT)** with HS256 algorithms and 24-hour expiration periods. Role-Based Access Control (RBAC) middleware strictly prevents patients from querying physician practice queues.

CHAPTER 04

Technology Stack & Core Modular Breakdown

Layer	Technology	Role in MedAssist AI
Frontend Client	React 18 + Vite	High-performance Single Page Application (SPA) with reactive component hierarchy.
Styling & UI	Tailwind CSS + Lucide Icons	Responsive clinical UI design with accessible medical indicator icons.
Backend Server	Python 3.12 + FastAPI	Asynchronous REST API with automatic data validation and processing.
AI / ML Engine	Scikit-Learn & TheFuzz	Random Forest disease classification and conversational NLP fuzzy matching.
Cloud Database	MongoDB Atlas Cloud	Cloud NoSQL persistence for patient histories, doctor slots, and symptom records.
Infrastructure	AWS EC2 + Nginx + Certbot	Ubuntu 24.04 LTS host with 24/7 systemd daemon and SSL/TLS HTTPS encryption.

4.2 Detailed Core Modules

Module 1: Authentication & Role-Based Access Control (RBAC)
Handles patient and doctor registration, bcrypt password hashing, and JWT bearer token issuance with role-gated API endpoints.

Module 2: Conversational NLP Fuzzy Symptom Tokenization
Transforms unstructured free-text into binary feature vectors by computing Levenshtein token similarity against 132 medical indicators.

Module 3: Dual-Pathway ML Clinical Prediction Engine
Runs Random Forest multi-class probabilistic inference for dataset conditions and evaluates heuristic emergency rules for rare/unlisted diseases.

Module 4: Multi-Factor Clinical Risk Stratification
Combines individual symptom severity weights, overall symptom count, and prediction confidence into four distinct risk classifications (Low, Moderate, High, Critical).

Module 5: Automated Clinical PDF Report Generation

Utilizes jsPDF and AutoTable to generate printable, standardized hospital consultation summaries complete with patient metadata and risk ratings.

Module 6: Specialist Directory & Telehealth Scheduling

Allows patients to browse verified doctors, filter by medical specialty, and reserve appointment slots stored in MongoDB Atlas.

Module 7: Physician Practice Analytics & Real-Time Triage Monitor

Provides healthcare providers with real-time patient queue metrics, triage risk badges, and appointment management dashboards.

CHAPTER 05

Dataset Engineering & Machine Learning Architecture

5.1 Dataset Composition & One-Hot Representation

The predictive engine was trained on a multi-dimensional clinical symptom matrix spanning **132 distinct symptoms** mapped to **41 diagnostic targets**. Input feature vectors are mapped into a 132-dimensional binary space:

```
x = [itching, skin_rash, continuous_sneezing, shivering, chills, joint_pain, ..., yellow_crust_ooze]
                                                ∈ {0, 1}^132
```

5.2 Random Forest Classifier Mathematical Theory

The classifier aggregates predictions from an ensemble of $N = 100$ de-correlated decision trees. Split points are determined by minimizing Gini Impurity:

$$I_G(p) = 1 - \sum (p_i)^2$$

The ensemble probability for predicted disease C given feature vector $\mathbf{x}$ is computed via:

$$P(C \mid x) = (1 / N) * \sum I(h_t(x) == C)$$

5.3 Levenshtein Fuzzy Matching NLP Formulation

To map unstructured colloquial user symptoms to standardized medical features, the NLP engine evaluates token similarity ratio $R(s_1, s_2)$:

$$R(s_1, s_2) = \left(\frac{|s_1| + |s_2| - lev(s_1, s_2)}{|s_1| + |s_2|} \right) * 100$$

Any user input scoring $\ge 75\%$ similarity is automatically converted to its canonical clinical indicator (e.g., "bad tummy ache" -> `abdominal_pain`).

CHAPTER 06

Implementation Milestones & Code Architecture

6.1 Development Lifecycle & Milestones

Milestone 1: Data Preprocessing & ML Model Serialization (.pkl)

Milestone 2: FastAPI REST Service & MongoDB Atlas Cloud Setup

Milestone 3: React 18 Single Page Architecture & Tailwind UI

Milestone 4: Role-Based Patient / Doctor Portals & Session Persistence

Milestone 5: AWS EC2 Cloud Hosting, Nginx Proxy & Let's Encrypt SSL

Milestone 6: End-to-End Verification & GitHub Branch Synchronization

6.2 Key Source Code Implementation Structure

```
backend/
├── main.py             # FastAPI REST endpoints & HTTP exception handlers
├── ml_engine.py        # Random Forest model loader, feature aligner & NLP tokenizer
├── database.py         # MongoDB Atlas client with PyMongo connection pooling & SSL
├── security.py         # Bcrypt password hashing & JWT token encoder/decoder
├── schemas.py          # Pydantic v2 data models with role & password validation
└── requirements.txt    # Python dependencies (fastapi, scikit-learn, pymongo, etc.)

frontend/
├── src/App.jsx         # Root router with localStorage refresh persistence
├── src/context/        # AuthContext (JWT session) & MedicalContext (Live DB state)
├── src/components/     # Patient, Doctor, Auth, and Common UI components
└── src/services/api.js # Asynchronous HTTP client communicating with backend
```

CHAPTER 07

User Roles & Interactive Portals Walkthrough

P Patient Portal Features

1. **Landing Page:** Clean healthcare introduction with role-based sign-in/register.
2. **Symptom Analysis:** Patient types conversational symptoms or selects tags.
3. **Predictive Diagnostics:** Top 3 differential conditions with confidence ratings and risk badges.
4. **PDF Export:** One-click generation of clinical summary report for hospital visits.
5. **Doctor Directory:** Real-time specialist booking with confirmed slots.

D Doctor / Provider Features

1. **Physician Sign-In:** Secure authentication loading verified credentials.
2. **Practice Analytics:** Live dashboard tracking total triage logs, high-risk cases, and bookings.
3. **Triage Queue Monitor:** Live table showing patient symptom profiles and computed risk levels.
4. **Appointment Manager:** Calendar slots and scheduled consultation management.

CHAPTER 08

Testing, Quality Assurance & Performance Evaluation

8.1 Machine Learning Classification Metrics

Clinical Condition Category	Precision	Recall	F1-Score
Infectious & Febrile Diseases (Malaria, Dengue, Typhoid)	**99.1%**	98.5%	**0.988**
Respiratory Illnesses (Pneumonia, Bronchial Asthma)	**98.2%**	97.8%	**0.980**
Gastrointestinal & Hepatic (GERD, Hepatitis, Jaundice)	**98.6%**	99.0%	**0.988**
Dermatological & Allergic (Fungal, Psoriasis, Impetigo)	**97.8%**	98.1%	**0.979**
Weighted Global Aggregate	**98.4%**	**98.4%**	**0.984**

8.2 End-to-End System Test Cases

TC-01: Conversational NLP Parsing: "fever with chills and throwing up" -> Maps to fever, chills, vomiting — PASSED ✅

TC-02: Unlisted Emergency Detection: "sharp right lower belly pain vomiting" -> Acute Appendicitis (Critical Risk) — PASSED ✅

TC-03: Session State Persistence: Page refresh (Cmd+R) preserves Doctor / Patient dashboard — PASSED ✅

TC-04: MongoDB Cloud Persistence: New patient registration instantly stores in cluster — PASSED ✅

CHAPTER 09

AWS Cloud Deployment & Production Results

AWS EC2 Production Configuration Summary ONLINE 🟢

Cloud Host:	Public IP:	Web Server:	SSL Protocol:
AWS EC2 (ap-south-1)	13.235.80.175	Nginx 1.28.3	Let's Encrypt TLSv1.3

9.2 Verified Live Endpoints

🌐 **Live Application URL:** https://medical-assist.duckdns.org (Active HTTPS SSL Padlock 🔒)

📦 **Internship Submission Branch:** akilandeshwari_s Branch on GitHub

CHAPTER 10

Conclusion & Future Roadmap

The **MedAssist AI** engineering project delivers a robust, verified, and cloud-hosted medical symptom analysis and disease prediction platform. By pairing an ensemble Random Forest machine learning classifier with conversational NLP fuzzy extraction, dual-pathway triage logic, and persistent MongoDB Atlas cloud storage, the system empowers patients with immediate triage guidance and furnishes physicians with actionable clinical data.

10.2 Future Engineering Roadmap

📱 IoT Wearable Telemetry

Real-time integration with smartwatches for SpO2, heart rate, and temperature streaming.

🌐 Multilingual Speech NLP

Voice recognition supporting regional Indian languages for non-English speaking patients.

📹 WebRTC Teleconsultation

Built-in peer-to-peer encrypted high-definition video consultations between doctor and patient.